

第36章 WWDG—窗口看门狗

本章参考资料：《STM32F4xx 中文参考手册》WWDG 章节。

学习本章时，配合《STM32F4xx 中文参考手册》WWDG 章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

36.1 WWDG 简介

STM32 有两个看门狗，一个是独立看门狗，一个是窗口看门狗。我们知道独立看门狗的工作原理就是一个递减计数器在 LSI 时钟的驱动下不断的往下递减计数，当减到 0 之前如果没有刷新递减计数器的值（即俗称的喂狗）的话，产生复位。窗口看门狗跟独立看门狗一样，也是一个递减计数器不断的往下递减计数，当减到一个固定值 0X40 时还不喂狗的话，产生复位，这个值叫窗口的下限，是固定的值，不能改变。这个是跟独立看门狗类似的地方，不同的地方是窗口看门狗的计数器的值在减到某一个数之前喂狗的话也会产生复位，这个值叫窗口的上限，上限值由用户独立设置。窗口看门狗计数器的值必须在上窗口和下窗口之间才可以喂狗，这就是窗口看门狗中窗口两个字的含义

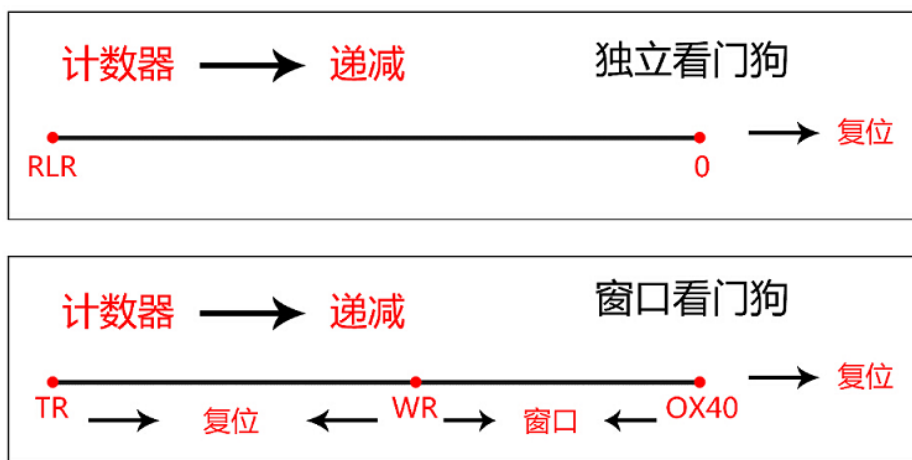


图 36-1 IWDG 与 WWDG 区别

RLR 是重装载寄存器，用来设置独立看门狗的计数器的值。TR 是窗口看门狗的计数器的值，由用户独立设置，WR 是窗口看门狗的上窗口值，由用户独立设置。

36.2 WWDG 功能框图剖析

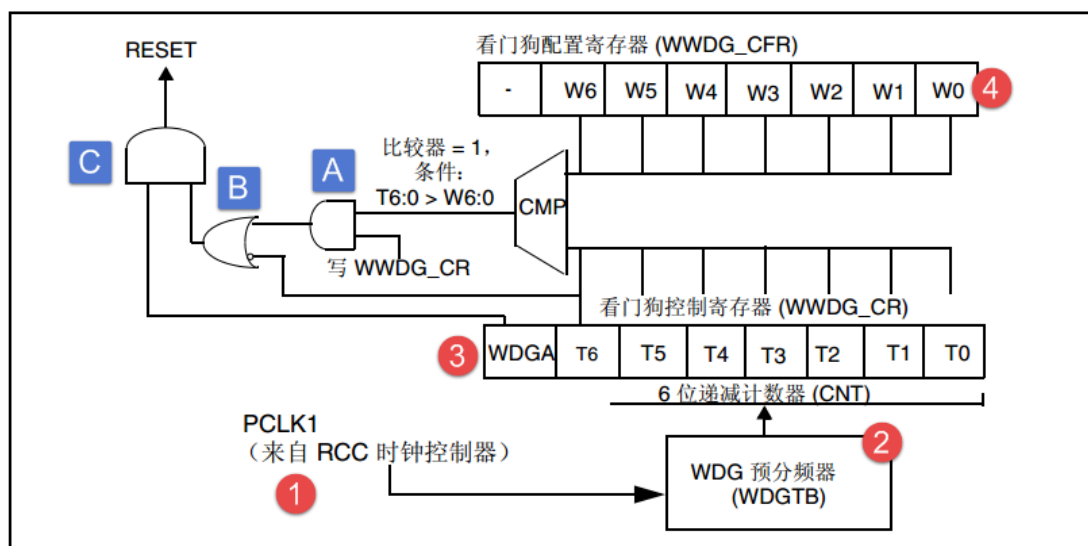


图 36-2 窗口看门狗功能框图

1. ①窗口看门狗时钟

窗口看门狗时钟来自 PCLK1，PCLK1 最大是 42M，由 RCC 时钟控制器开启。

2. ②计数器时钟

计数器时钟由 CK 计时器时钟经过预分频器分频得到，分频系数由配置寄存器 CFR 的位 8:7 WDTB[1:0]配置，可以是[0,1,2,3]，其中 CK 计时器时钟=PCLK1/4096，除以 4096 是手册规定的，没有为什么。所以计数器的时钟 $CNT_CK = PCLK1 / 4096 / (2^{WDGTB})$ ，这就可以算出计数器减一个数的时间 $T = 1 / CNT_CK = T_{pclk1} * 4096 * (2^{WDGTB})$ 。

3. ③计数器

窗口看门狗的计数器是一个递减计数器，共有 7 位，其值存在控制寄存器 CR 的位 6:0，即 T[6:0]，当 7 个位全部为 1 时是 0X7F，这个是最大值，当递减到 T6 位变成 0 时，即从 0X40 变为 0X3F 时候，会产生看门狗复位。这个值 0X40 是看门狗能够递减到的最小值，所以计数器的值只能是：0X40~0X7F 之间，实际上用来计数的是 T[5:0]。当递减计数器递减到 0X40 的时候，还不会马上产生复位，如果使能了提前唤醒中断：CFR 位 9 EWI 置 1，则产生提前唤醒中断，如果真进入了这个中断的话，就说明程序肯定是出问题了，那么在中断服务程序里面我们就需要做最重要的工作，比如保存重要数据，或者报警等，这个中断我们也叫它死前中断。

4. ④窗口值

我们知道窗口看门狗必须在计数器的值在一个范围内才可以喂狗，其中下窗口的值是固定的 0X40，上窗口的值可以改变，具体的由配置寄存器 CFR 的位 6:0 W[6:0]设置。其值必须大于 0X40，如果小于或者等于 0X40 就是失去了窗口的价值，而且也不能大于计数器的值，所以必须得小于 0X7F。那窗口值具体要设置成多大？这个得根据我们需要监控的程序运行时间来决定。如果我们要监控的程序段 A 运行的时间为 T_a ，当执行完这段程序之后就要进行喂狗，如果在窗口时间内没有喂狗的话，那程序就肯定是出问题了。一般计数器的值 TR 设置成最大 0X7F，窗口值为 WR，计数器减一个数的时间为 T，那么时间： $(TR-WR)*T$ 应该稍微大于 T_a 即可，这样就能做到刚执行完程序段 A 之后喂狗，起到监控的作用，这样也就可以算出 WR 的值是多少。

5. ⑤计算看门狗超时时间

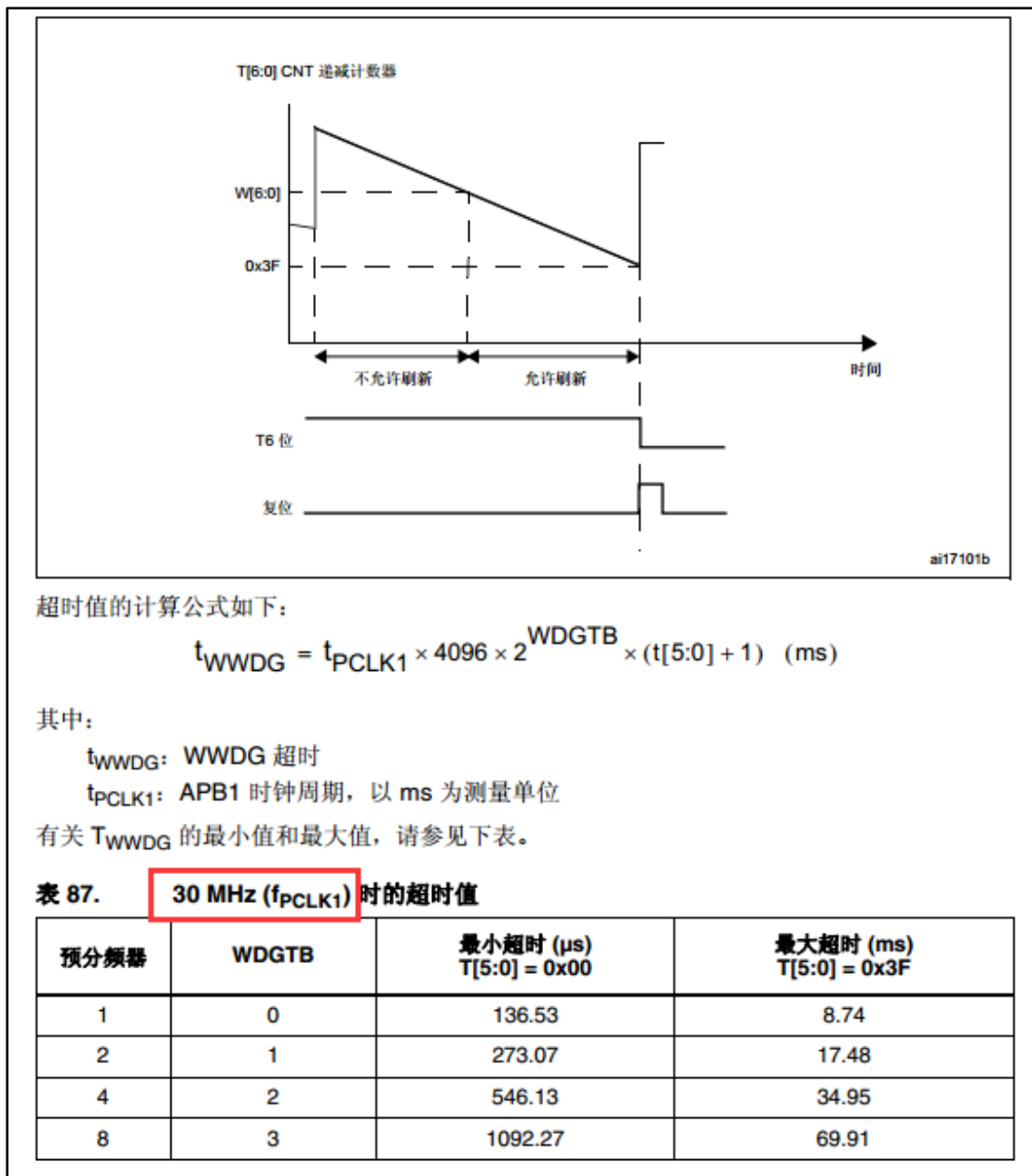


图 36-3 窗口看门狗时序图

这个图来自数据手册，从图我们知道看门狗超时时间： $T_{\text{wwdg}} = T_{\text{pclk1}} \times 4096 \times 2^{\text{wdgtb}} \times (T[5:0] + 1) \text{ ms}$ ，当 PCLK1 = 30MHz 时，WDGTB 取不同的值时有最小和最大的超时时间，那这个最小和最大的超时时间该怎么理解，又是怎么算出来的？讲起来有点绕，这里我稍微讲解下 WDTB=0 时是怎么算的。递减计数器有 7 位 T[6:0]，当位 6 变为 0 的时候就会产生复位，实际上有效的计数位是 T[5:0]，而且 T6 必须先设置为 1。如果 T[5:0]=0 时，递减计数器再减一次，就产生复位了，那这减一的时间就等于计数器的周期 $= 1/\text{CNT_CK} = T_{\text{pclk1}} \times 4096 \times (2^{\text{WDGTB}}) = 1/30 \times 4096 \times 2^0 = 136.53\mu\text{s}$ ，这个就是最短的

超时时间。如果 T[5:0]全部装满为 1，即 63，当他减到 0X40 变成 0X3F 时，所需的时间就是最大的超时时间= $113.7 \times 2^5 = 136.53 \times 64 = 8.74\text{ms}$ 。同理，当 WDG TB 等于 1/2/3 时，代入公式即可。

36.3 怎么用 WWDG

WWDG 一般被用来监测，由外部干扰或不可预见的逻辑条件造成的应用程序背离正常的运行序列而产生的软件故障。比如一个程序段正常运行的时间是 50ms，在运行完这个段程序之后紧接着进行喂狗，如果在规定的窗口内还没有喂狗，那就说明我们监控的程序出故障了，跑飞了，那么就会产生系统复位，让程序重新运行。

36.4 WWDG 喂狗实验

36.4.1 硬件设计

4、WWDG 一个

5、LED 两个

WWDG 属于单片机内部资源，不需要外部电路，需要两个 LED 来指示程序的运行状态。

36.4.2 软件设计

我们编写两个 WWDG 驱动文件，bsp_wwdg.h 和 bsp_wwdg.c，用来存放 WWDG 的初始化配置函数。

1. 代码分析

这里只讲解核心的部分代码，有些变量的设置，头文件的包含等并没有涉及到，完整的代码请参考本章配套的工程。

WWDG 配置函数

代码 36-1 WWDG 配置函数

```
1 /* WWDG 配置函数
2  * tr : 递减计时器的值， 取值范围为：0x7f~0x40
3  * wr : 窗口值，取值范围为：0x7f~0x40
4  * prv: 预分频器值，取值可以是
```

```
5  *      @arg WWDG_Prescaler_1: WWDG counter clock = (PCLK1/4096)/1
6  *      @arg WWDG_Prescaler_2: WWDG counter clock = (PCLK1/4096)/2
7  *      @arg WWDG_Prescaler_4: WWDG counter clock = (PCLK1/4096)/4
8  *      @arg WWDG_Prescaler_8: WWDG counter clock = (PCLK1/4096)/8
9  */
10 void WWDG_Config(uint8_t tr, uint8_t wr, uint32_t prv)
11 {
12     // 开启 WWDG 时钟
13     RCC_APB1PeriphClockCmd(RCC_APB1Periph_WWDG, ENABLE);
14
15     // 设置递减计数器的值
16     WWDG_SetCounter( tr );
17
18     // 设置预分频器的值
19     WWDG_SetPrescaler( prv );
20
21     // 设置上窗口值
22     WWDG_SetWindowValue( wr );
23
24     // 设置计数器的值, 使能 WWDG
25     WWDG_Enable(WWDG_CNT);
26
27     // 清除提前唤醒中断标志位
28     WWDG_ClearFlag();
29     // 配置 WWDG 中断优先级
30     WWDG_NVIC_Config();
31     // 开 WWDG 中断
32     WWDG_EnableIT();
33 }
```

WWDG 配置函数有三个形参, **tr** 是计数器的值, 一般我们设置成最大 0X7F, **wr** 是上窗口的值, **tr** 和 **wr** 之间的计数器计数的时间就是我们需要监控的程序运行时间, 所以 **wr** 的值我们要根据监控的程序运行时间来设置, 但是值必须在 0X40 和计数器的值之间, **prv** 用来设置预分频的值, 取值具体见代码清单 36-1。

代码清单 36-1 形参 prv 取值

```
1 /*
2  *      @arg WWDG_Prescaler_1: WWDG counter clock = (PCLK1/4096)/1
3  *      @arg WWDG_Prescaler_2: WWDG counter clock = (PCLK1/4096)/2
4  *      @arg WWDG_Prescaler_4: WWDG counter clock = (PCLK1/4096)/4
5  *      @arg WWDG_Prescaler_8: WWDG counter clock = (PCLK1/4096)/8
6  */
```

这些宏在 `stm32f4xx_wwdg.h` 中定义, 宏展开是 32 位的 16 进制数, 具体作用是设置配置寄存器 CFR 的位 8:7 WDGTCR[1:0], 获得各种分频系数, 最终确定驱动计数器的时钟。

代码清单 36-2 WWDG 中断优先级函数

```
1 // WWDG 中断优先级初始化
2 static void WWDG_NVIC_Config(void)
3 {
4     NVIC_InitTypeDef NVIC_InitStructure;
5
6     NVIC_PriorityGroupConfig(NVIC_PriorityGroup_1);
7     NVIC_InitStructure.NVIC_IRQChannel = WWDG_IRQn;
8     NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
9     NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
10    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
11    NVIC_Init(&NVIC_InitStructure);
12 }
```

在递减计数器减到 0X40 的时候，我们开启了提前唤醒中断，这个中断我们称它为死前中断或者叫遗嘱中断，在中断函数里面我们应该出来最重要的事情，而且必须得快，因为递减计数器再减一次，就会产生系统复位。

提前唤醒中断复位程序

代码清单 36-3 提前唤醒中断服务程序

```
1 // WWDG 中断复服务程序，如果发生了此中断，表示程序已经出现了故障，
2 // 这是一个死前中断。在此中断服务程序中应该干最重要的事，
3 // 比如保存重要的数据等，这个时间具体有多长，要
4 // 由 WDGTB 的值决定：
5 // WDGTB:0    113us
6 // WDGTB:1    227us
7 // WDGTB:2    455us
8 // WDGTB:3    910us
9 void WWDG_IRQHandler(void)
10 {
11     // 清除中断标志位
12     WWDG_ClearFlag();
13
14     // LED2 亮，点亮 LED 只是示意性的操作，
15     // 真正使用的时候，这里应该是做最重要的事情
16     LED2(ON);
17 }
```

喂狗函数

代码清单 36-4 喂狗函数

```
1 // 喂狗
2 void WWDG_Feed(void)
3 {
4     // 喂狗，刷新递减计数器的值，设置成最大 WDG_CNT=0X7F
5     WWDG_SetCounter( WWDG_CNT );
6 }
```

喂狗就是重新刷新递减计数器的值防止系统复位，喂狗一般是在主函数中喂。

主函数

代码 36-2 主函数

```
1 #include "stm32f4xx.h"
2 #include "../led/bsp_led.h"
3 #include "../wwdg/bsp_wwdg.h"
4
5 static void Delay(__IO u32 nCount);
6
7 /**
8  * @brief 主函数
9  * @param 无
10  * @retval 无
11  */
12
13 int main(void)
14 {
15     uint8_t wwdg_tr, wwdg_wr;
16
17     /* LED 端口初始化 */
18     LED_GPIO_Config();
```

```
19
20 // BLUE 蓝色灯亮
21 LED3(ON);
22 Delay(0xFFFFFFFF);
23
24 // WWDG 配置
25
26 /* WWDG 配置函数
27  * tr : 递减计时器的值, 取值范围为: 0x7f~0x40, 超出范围会直接复位
28  * wr : 窗口值, 取值范围为: 0x7f~0x40
29  * prv: 预分频器值, 取值可以是
30  * @arg WWDG_Prescaler_1: WWDG counter clock = (PCLK1(42MHz)/4096)/1
31  * 约 10253Hz 97us
32  * @arg WWDG_Prescaler_2: WWDG counter clock = (PCLK1(42MHz)/4096)/2
33  * 约 5126Hz 195us
34  * @arg WWDG_Prescaler_4: WWDG counter clock = (PCLK1(42MHz)/4096)/4
35  * 约 2563Hz 390us
36  * @arg WWDG_Prescaler_8: WWDG counter clock = (PCLK1(42MHz)/4096)/8
37  * 约 1281Hz 780us
38  *
39  * 例: tr = 127(0x7f, tr 的最大值)
40  * wr = 80 (0x50, 0x40 为最小 wr 最小值)
41  * prv = WWDG_Prescaler_8, 这是计数器计一次的时间为 780us
42  * 则有: 780 * (127-80) = 36.6ms < 刷新窗口 < ~780 * 64 = 49.9ms
43  * 也就是说调用 WWDG_Config 进行这样的配置, 若在之后的 36.6ms 前喂狗, 系统会复位,
44  * 在 49.9ms 后没有喂狗, 系统也会复位。
45  * 需要在刷新窗口的时间内喂狗, 系统才不会复位。
46  */
47 // 初始化 WWDG: 配置计数器初始值, 配置上窗口值, 启动 WWDG, 使能提前唤醒中断
48 WWDG_Config(127,80,WWDG_Prescaler_8);
49
50 // 窗口值我们在初始化的时候设置成 0x5F, 这个值不会改变
51 wwdg_wr = WWDG->CFR & 0X7F;
52
53 while (1) {
54     // BLUE 蓝色灯
55     LED3(OFF);
56     //-----
57     // 这部分应该写需要被 WWDG 监控的程序, 这段程序运行的时间
58     // 决定了窗口值应该设置成多大。
59     //-----
60
61     // 计时器值, 初始化成最大 0x7F, 当开启 WWDG 时候, 这个值会不断减小
62     // 当计数器的值大于窗口值时喂狗的话, 会复位, 当计数器减少到 0x40
63     // 还没有喂狗的话就非常非常危险了, 计数器再减一次到了 0x3F 时就复位
64     // 所以要当计数器的值在窗口值和 0x40 之间的时候喂狗, 其中 0x40 是固定的。
65     wwdg_tr = WWDG->CR & 0X7F;
66
67     if ( wwdg_tr < wwdg_wr ) {
68         // 喂狗, 重新设置计数器的值为最大 0x7F
69         WWDG_Feed();
70     }
71 }
72 }
```

主函数中我们把 WWDG 的计数器的值设置为 0x7F, 上窗口值设置为 0x5F, 分频系数为 8 分频, 则计数器减 1 的时间约为 728us。在 while 死循环中, 我们不断读取计数器的值, 当计数器的值减小到小于上窗口值的时候, 我们喂狗, 让计数器重新计数。

在 while 死循环中，一般是我们需要监控的程序，这部分代码的运行时间，决定了上窗口值应该设置为多少，当监控的程序运行完毕之后，我们需要执行喂狗程序，比起独立看门狗，这个喂狗的窗口时间是非常短的，对时间要求很精确。如果没有在这个窗口时间内喂狗的话，那就说明程序出故障了，会产生提前唤醒中断，最后系统复位。

36.4.3 下载验证

把编译好的程序下载到开发板，LED3 被点亮，一段时间之后熄灭，之后 LED3 一直就没有被点亮过，说明系统没有产生复位，如果产生复位的话 LED3 会再被点亮一次。中断服务程序中的 LED 也没被点亮过，说明喂狗正常。

